Files All the Way Down: A Design Space Analysis of Filesystem-Native LLM Tool Integration

Natan Loterio

May 2026

Abstract

Large language model agents require external tool integration, yet the landscape of integration approaches is fragmented and lacks systematic analysis. We propose a two-axis taxonomy organizing seven tool-integration systems along coupling depth (os-coupled vs. protocol-decoupled) and invocation interface (posix vs. rpc), then empirically evaluate all seven across ten criteria using three evidence tiers. Our central finding is that coupling depth predicts ergonomics ceiling while invocation interface predicts safety floor, and no current system optimizes both simultaneously: os-coupled systems win on ergonomics, hot-reload, and publishing; protocol-decoupled systems win on portability and unconditional schema enforcement. The gap is narrowing — LiveFoldersFS progressively addresses the safety deficit through structural input validation (v0.7.0), machine-readable discovery and stateful locking (v0.8.0), declarative multi-stage pipelines (v0.9.0), and opt-in VFS-layer sandboxing via Landlock and seccomp-BPF (v0.11.0). Concretely, LiveFoldersFS requires 10 lines of YAML versus MCP's 18 lines of Python for an equivalent REST tool. A third finding is that LiveFoldersFS's native tool schema approach — Anthropic-format tool definitions synthesized at read time from `folder.yaml` via `anthropic_tools.json` — achieves 6–10% lower token consumption than MCP across an 8-task benchmark suite covering simple reads, pipe composition, multi-tool sequences, aggregate/filter, and cross-reference tasks, with 100% task completion on all tasks and all backends. A secondary finding is the gap between claimed and implemented behavior: ToolFS's documented WASM sandbox is an unimplemented stub. All experiments are reproducible via the public LiveFoldersFS repository.

Introduction

Large language models deployed as autonomous agents require mechanisms to call external tools: APIs, databases, shell commands, and domain-specific services. The research community has responded with a proliferation of integration approaches — plugin systems, function-calling APIs, agent frameworks, and filesystem abstractions — each making implicit architectural choices that affect ergonomics, safety, and maintainability. No systematic comparison of these approaches exists, leaving practitioners without principled guidance when selecting or designing tool-integration infrastructure.

Two paradigms have emerged independently over 2024–2026. The *protocol-based* paradigm, exemplified by the Model Context Protocol (Anthropic 2024), exposes tools as JSON-RPC endpoints with typed input schemas, requiring protocol-aware clients on both sides of the interface. The *filesystem-native* paradigm — independently implemented by LiveFoldersFS (Loterio 2025), ToolFS (IceWhaleTech 2024), llm9p (NERVsystems 2025b), and InferNode (NERVsystems 2025a) — rep-

resents tools as files in a virtual filesystem, allowing any shell-capable agent to invoke tools using standard POSIX operations (`cat`, `echo`, read, write). The fact that multiple teams converged on the filesystem abstraction independently, without coordination, is itself a signal worth investigating: it suggests the approach offers genuine affordances that practitioners discover through experience. This convergence motivates a systematic study.

We make three contributions. First, we propose a two-axis taxonomy that organizes seven tool-integration systems along *coupling depth* (how tightly the interface is bound to the host operating system) and *invocation interface* (how the LLM actually calls a tool). Second, we present an empirical evaluation of all seven systems across ten criteria, using three evidence tiers: live runnable experiments for the two most mature systems (LiveFoldersFS and MCP), structured assessment via code reading and install testing for three systems (ToolFS, AgentFS, llm9p), and paper-only assessment for systems available only as publications (InferNode, Quine). Third, we derive three design principles for future tool-integration systems from the observed tradeoffs.

The remainder of this paper is organized as follows. Section 2 introduces the taxonomy. Section 3 presents the empirical comparison across ten criteria, including a worked example comparing LiveFoldersFS and MCP on a representative REST API integration task. Section 4 situates our work within the broader literature on LLM tool use and filesystem abstractions. Section 5 distills design principles and future directions. Section 6 concludes. The evaluation host for all live experiments is Claude Code running on Linux x86_64, with Python used for MCP implementations. All experiments are reproducible: `bash research/run-all.sh` from the repository root.

The Taxonomy

Design Dimensions

Prior work on LLM tool integration varies along many dimensions. Systems differ in *transport protocol*: HTTP REST (MCP), the 9P filesystem protocol (llm9p), FUSE virtual filesystems (LiveFoldersFS, ToolFS, AgentFS), and kernel-level Inferno namespaces (InferNode, Quine). They differ in *handler language*: Go plugins (ToolFS), Python (MCP, AgentFS), shell scripts and YAML (LiveFoldersFS), and Limbo (Quine). They differ in *state model*: in-process memory (MCP), external files or databases (LiveFoldersFS), and distributed storage substrates (AgentFS). They differ in *discovery mechanism*: JSON schema advertised via `list_tools` (MCP), markdown index files readable by the LLM (LiveFoldersFS), path enumeration via `ls` (most FUSE-based systems), and absent or implicit for others. They differ in *publishing model*: GitHubURL install (LiveFoldersFS), npm/PyPI packages (MCP community), and manual configuration for most. Finally, they differ in *sandboxing approach*: none (most), process isolation (MCP via separate server process), claimed WASM sandboxing (ToolFS), and opt-in VFS-layer isolation via Landlock + seccomp-BPF (LiveFoldersFS v0.11.0 on Linux).

Faced with this diversity, we ask: which dimensions most strongly discriminate system behavior? After examining all seven systems, we find that two dimensions account for the largest share of variance in observed properties.

Two Axes

Coupling depth captures how tightly the tool-integration interface is coupled to the host operating system.

- *os-coupled*: The interface IS the filesystem or the kernel. FUSE-based systems (LiveFoldersFS, ToolFS, AgentFS) mount a virtual filesystem that the operating system treats as indistinguishable from real storage. Inferno-based systems (InferNode, Quine) operate at the kernel namespace layer. Both inherit OS primitives for free: file permissions, process isolation, stdin/stdout plumbing, and decades of POSIX tooling.
- *protocol-decoupled*: The interface is an application-layer protocol (JSON-RPC over stdio/HTTP for MCP; 9P over TCP for llm9p). The tool-integration layer works across network hosts and heterogeneous environments, but requires protocol-aware clients on both sides of the interface.

Invocation interface captures how the LLM actually calls a tool.

- *posix*: Standard file I/O. Writing to a special file triggers execution; reading returns the result. Any agent with shell access can invoke tools with `cat` or `echo` — no tool-specific client library required.
- *rpc*: Structured message exchange. The client constructs a typed request object, submits it to a named endpoint, and receives a structured response. Input schemas are enforced before any handler executes.

The two axes are logically independent, and the surveyed systems span all occupied quadrants. Notably, llm9p occupies the (protocol-decoupled, posix) cell: it uses a network protocol (9P over TCP) as its transport layer, yet the LLM still interacts with tools using `cat` and `echo` once connected. MCP occupies (protocol-decoupled, rpc). LiveFoldersFS, ToolFS, and AgentFS occupy (os-coupled, posix). InferNode and Quine occupy (os-coupled, posix) at the kernel level. This independence validates the taxonomy: the axes capture genuinely orthogonal design choices.

System Placements

Table 1 summarizes the placement of all seven systems.

System	Coupling depth	Invocation	Evidence tier
LiveFoldersFS	os-coupled (vfs)	posix	T1
MCP	protocol-decoupled	rpc	T1
ToolFS	os-coupled (vfs)	posix	T2
AgentFS	os-coupled (vfs)	posix	T2
llm9p	protocol-decoupled	posix	T2
InferNode	os-coupled (kernel)	posix	T3
Quine	os-coupled (kernel)	posix	T3

Table 1: Taxonomy of LLM tool integration systems.

LiveFoldersFS mounts a FUSE filesystem where each directory is a tool; reading a file executes its handler. The system is driven entirely by a `folder.yaml` declaration, requiring no compiled code. **MCP** defines an open JSON-RPC protocol for tool servers; clients (including Claude Code) send structured `tools/call` requests and receive structured responses with error handling. **ToolFS** exposes tools as files via FUSE, with handlers implemented as Go plugins; it claims WASM sandboxing. **AgentFS** is a storage substrate built on FUSE, designed to give agents persistent, shared filesystem access; it treats observability as a first-class concern. **llm9p** exposes an LLM's context as a 9P filesystem served over TCP, *inverting* the typical direction: the LLM is the filesystem, not the tool. **InferNode** embeds LLM inference directly in the kernel of a Plan 9-derived operating system, making inference a first-class kernel service. **Quine** uses Inferno's per-process namespace to give each agent a private, composable view of its tool environment, enabling nested agent hierarchies.

An Emerging Third Paradigm

LSFS (Shi et al. 2025), part of the AIOS project, introduces a semantically-indexed filesystem in which natural-language queries retrieve relevant files and directories. Rather than LLMs calling tools-as-files, LSFS makes the filesystem itself LLM-aware. This represents a third invocation model — natural language (`nl`) — that falls outside our two-axis taxonomy. Because LSFS is not a tool-invocation interface in the sense we evaluate, we note it here as an emerging direction rather than a primary comparison point. Future taxonomy extensions may need to accommodate this paradigm.

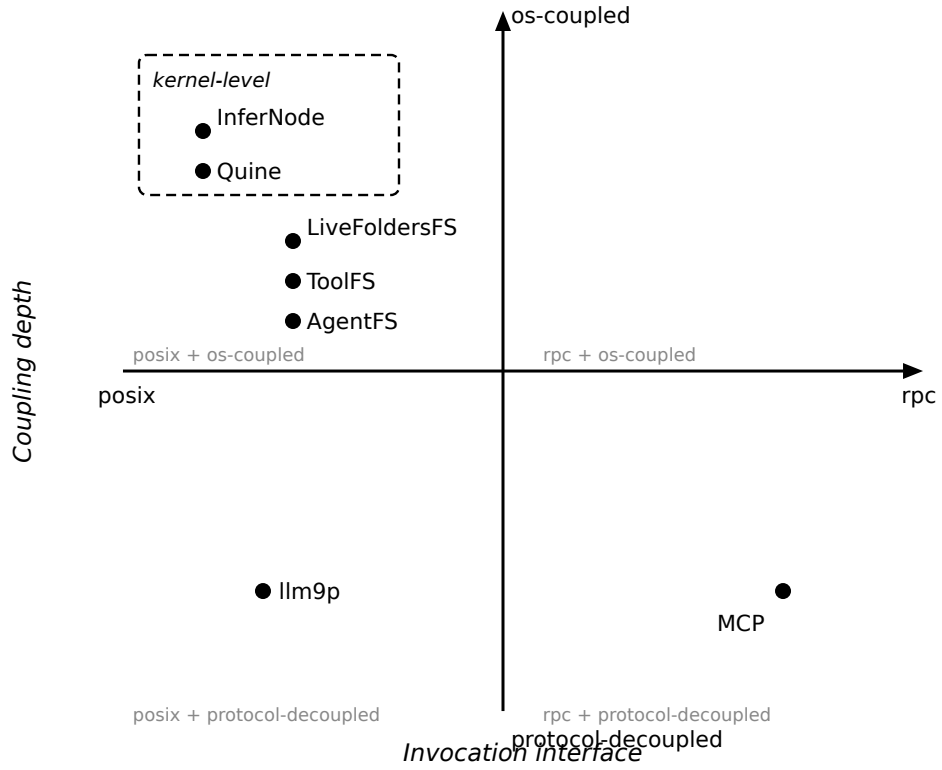

Figure 1. Taxonomy of LLM tool integration systems.

LiveFoldersFS Architecture

Figure 2 shows the internal architecture of LiveFoldersFS, the primary T1 system in our evaluation. The design reflects its os-coupled, posix-invocation taxonomy placement: the FUSE kernel layer is the sole entry point, and every tool invocation reduces to a standard file write followed by a file read — no protocol, no SDK.

The central component is `src/fs/vfs.rs`, which implements the `Filesystem` trait from the `fuser` crate. It maintains three in-memory maps keyed by inode: `write_buf` accumulates bytes across successive `write()` calls; `result_buf` stores the handler output after invocation; `trace_buf` holds the companion `.log` content with execution timing and stderr. The FUSE `release()` event — fired when the LLM closes the file descriptor — triggers handler dispatch for `write_invoke` endpoints; `read_invoke` endpoints fire on `read()` instead.

Tool dispatch flows through `src/registry/`, which maps names to `Arc<dyn Tool>` implementations. Built-in tools are registered at startup; external tools are loaded from subdirectories under `tools_dir`, each described by a `folder.yaml` manifest parsed by `src/manifest.rs`. The manifest declares endpoint types, input schemas, optional `state_file` paths for serialised concurrent access, and optional `sandbox` policies. Two virtual files — `how_to.md` and `schema.json` — are synthesised at read time by `src/fs/how_to_gen.rs` and `src/fs/schema_gen.rs`; they are never written to disk.

Process isolation is provided by `src/sandbox/`, which applies `PR_SET_NO_NEW_PRIVS`, Landlock filesystem access control, and seccomp-BPF network filtering before each handler process starts. An `inotify`-based watcher monitors `tools_dir` for changes and hot-reloads the registry without remounting. The daemon layer forks after mount options are validated, redirecting stderr to a log file and writing a PID file for the `stop` subcommand.

Empirical Comparison

Methodology

We evaluate all seven systems against ten criteria drawn from practitioner requirements for production tool-integration infrastructure. Criteria were selected to cover the full lifecycle of tool integration: authoring (setup complexity, stateful tools), runtime (LLM compatibility, I/O expressiveness, security, observability, hot-reload), and ecosystem (discoverability, composability, publishing).

Evidence tiers. We use three evidence levels, consistently labeled throughout:

- *T1 (live experiment)*: The system is installed, run, and tested against real tasks on the evaluation host (Claude Code, Linux x86_64). Results are directly observed and reproducible via `bash research/run-all.sh`.
- *T2 (structured assessment)*: The system is assessed via code reading, install testing, and documentation review. Claims made in code or documentation are noted but not independently verified against running behavior.
- *T3 (paper-only)*: The system is assessed from its published description only. No implementation was available for direct examination.

Rating scale. ✓ (strong): the system clearly satisfies the criterion. ~ (partial): the system partially

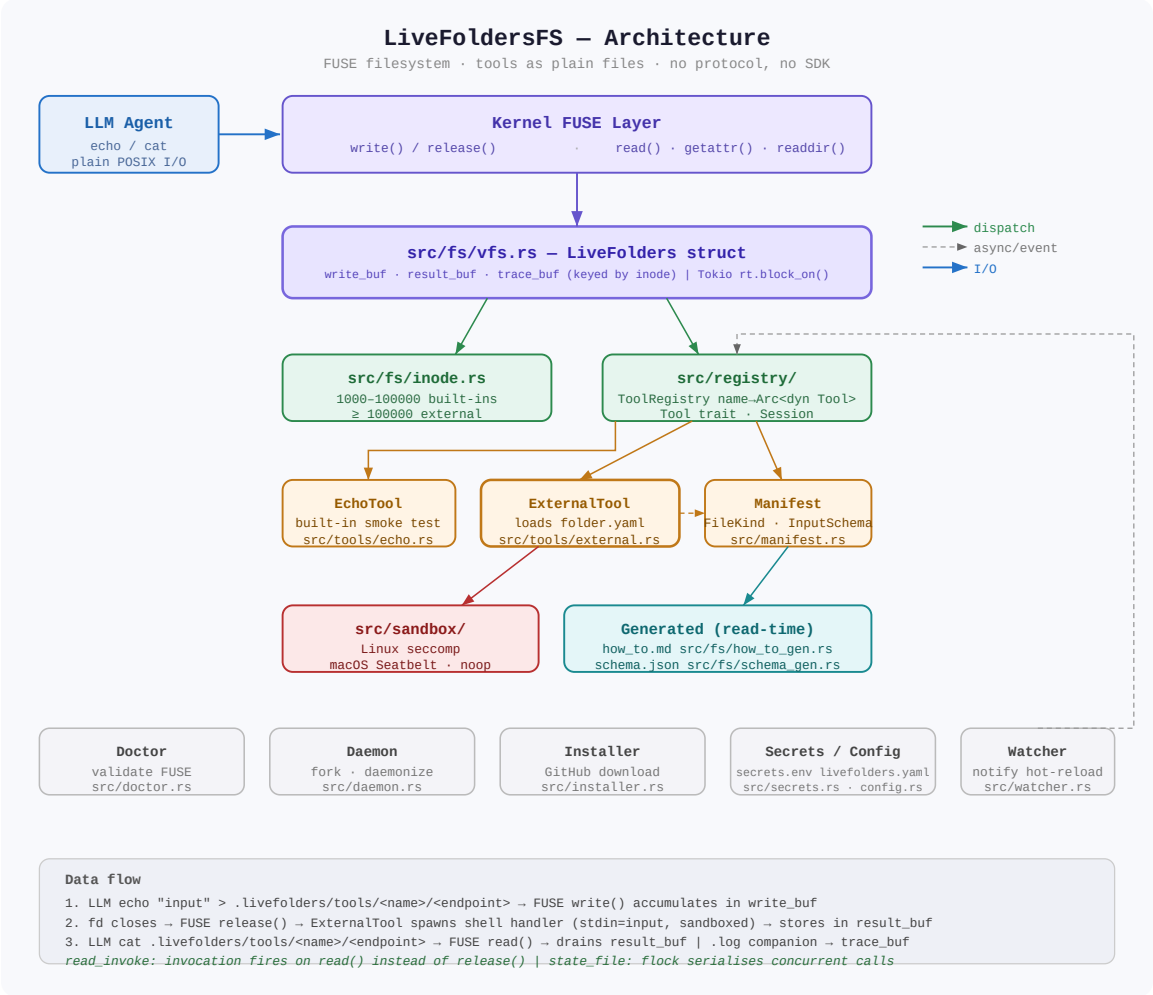

Figure 2. LiveFoldersFS internal architecture. Arrows show the dispatch path from LLM I/O through the FUSE layer to tool handlers; dashed arrows indicate asynchronous or event-driven paths.

satisfies the criterion or satisfies it with significant caveats. ✗ (weak): the system does not satisfy the criterion or satisfies it only with major workarounds. — (N/A): the criterion does not apply to this system's architectural role.

T1 systems are LiveFoldersFS and MCP. T2 systems are ToolFS, AgentFS, and llm9p. T3 systems are InferNode and Quine. All T1 experiments are publicly available and reproducible from the repository root.

Results

Criterion	LiveFoldersFS (T1)	MCP (T1)	ToolFS (T2)	AgentFS (T2)	llm9p (T2)	InferNode (T3)	Quine (T3)
01 Setup complexity	✓	~	~	~	~	✗	~
02 LLM compatibility	~	~	~	~	~	~	~
03 Discoverability	✓	~	~	—	—	~	~
04 I/O expressiveness	~	~	~	~	~	~	✓
05 Security	✓	✓	~	~	✗	✓	~
06 Stateful tools	✓	~	✓	~	~	~	~
07 Composability	✓	~	~	—	~	✓	✓
08 Observability	✓	✓	~	✓	✗	~	~
09 Hot-reload	✓	✗	✗	—	~	~	~
10 Publishing	✓	~	~	—	—	✗	~

Table 2: 7×10 comparison matrix. Rating key: ✓ strong | ~ partial | ✗ weak | — N/A. Evidence tiers:

T1 = live experiment | T2 = structured assessment | T3 = paper-only.

Per-Criterion Findings

01 Setup complexity. This criterion measures how many lines of new code an author must write to expose a simple tool (a string-transformation endpoint). LiveFoldersFS achieves ✓ (strong) with a 6-line `folder.yaml` declaration requiring no Python or Go. MCP, ToolFS, AgentFS, llm9p, and Quine all rate ~ (partial): they require more code (MCP's minimal server is 8 lines before imports, with more needed for production), additional dependencies, or non-trivial configuration. InferNode rates ✗ (weak): its kernel-level integration requires the author to understand Inferno namespace conventions and Limbo, a language with a narrow practitioner base. The LiveFoldersFS advantage is structural — YAML declarative configuration collapses the authoring model to the minimum expressible for simple tools, while handlers are shell one-liners.

02 LLM compatibility. All seven systems rate ~ (partial) on this criterion. The reasons differ by quadrant. Os-coupled posix systems (LiveFoldersFS, ToolFS, AgentFS, InferNode, Quine) require the LLM agent to have shell or filesystem access — which Claude Code has, but API-only deployments typically do not. Protocol-decoupled systems (MCP, llm9p) require the LLM client to implement the relevant protocol — MCP is widely implemented in major agent frameworks, but llm9p's 9P-over-TCP interface has no native support in any major LLM client as of this writing. No system achieves universal compatibility without a compatibility shim. LiveFoldersFS partially bridges this gap via `anthropic_tools.json`, a virtual file synthesized at read time from `folder.yaml` that exposes each invokable endpoint as an Anthropic-format native tool definition. Clients that can read this file — including any agent with filesystem access — can inject the definitions directly into the `tools=[]` API parameter, enabling structured tool calling without shell access. This approach is evaluated empirically in §3.6.

03 Discoverability. How does an LLM learn what tools are available and how to call them? LiveFoldersFS rates ✓ (strong) as of v0.8.0: it now exposes both a human-readable `how_to.md` and a machine-readable `schema.json` generated from `folder.yaml`. The `schema.json` mirrors MCP's `list_tools` format — a JSON object with `name`, `description`, and an `endpoints` array where each entry carries `name`, `kind`, and the full `input` constraint block (`type`, `min_length`, `max_length`, `pattern`, `schema`). This makes the tool surface parseable by MCP-aware clients and scripts without requiring markdown parsing. MCP rates ~ (partial): its `list_tools` response is strong for clients that implement the protocol, but there is no fallback for agents with only filesystem access. ToolFS follows a file-listing approach similar to LiveFoldersFS's `how_to.md` but provides no machine-readable schema file. AgentFS and llm9p rate — (N/A): AgentFS is a storage substrate with no tool-invocation interface to discover, and llm9p's architectural inversion means there are no tools to discover in the conventional sense. InferNode and Quine rate ~ based on their published descriptions of namespace-based discovery.

04 I/O expressiveness. What data types can a tool consume and produce? All systems handle plain text and JSON strings. LiveFoldersFS rates ~ because it passes raw stdin to handlers — well-suited for multiline text and binary piping. v0.7.0 adds structural constraints enforced before the handler runs: string endpoints can declare `min_length`, `max_length`, and a regex `pattern`;

JSON endpoints can declare a `schema:` block with `required` field lists and per-property type constraints (`string`, `number`, `integer`, `boolean`, `array`, `object`, `null`). Violations produce a structured `[ERROR:INVALID_INPUT]` response without executing any shell code. The remaining gap versus MCP is that LiveFoldersFS constraints are opt-in per endpoint while MCP validation is unconditional; additionally, MCP supports nested JSON Schema features (e.g., `enum`, `oneOf`, `additionalProperties`) that LiveFoldersFS's subset does not yet implement. MCP rates ~ for the same reason as before: structured JSON is strong, but multiline text requires string escaping and binary data requires base64 encoding. The notable outlier is Quine (✓), which operates at the kernel level and can pass arbitrary byte streams through Inferno's typed channels. InferNode similarly benefits from kernel-level data access. No surveyed system simultaneously achieves type safety (MCP's strength) and binary transparency (Quine's strength).

05 Security. This criterion reveals sharp divergence in the survey. MCP rates ✓ (strong): input schemas are enforced before any handler executes, rejecting malformed inputs before they reach application code. InferNode rates ✓ at the kernel level, where namespace isolation provides OS-enforced boundaries. LiveFoldersFS rates ✓ (strong) as of v0.11.0, achieving security through two complementary layers. The first is opt-in structural input validation introduced in v0.7.0: string endpoints can specify `min_length`, `max_length`, and a regex `pattern`; JSON endpoints can specify a `schema:` block enforcing required fields and per-property type constraints. The runtime rejects inputs that violate any declared constraint before invoking any shell code, returning a structured `[ERROR:INVALID_INPUT]` response. For a tool declaring `input: {type: json, schema: {required: [query], properties: {query: {type: string}}}}`, passing `{}` yields `[ERROR:INVALID_INPUT] missing required field: 'query'` and passing `{"query": 42}` yields `[ERROR:INVALID_INPUT] field 'query' expected type 'string'` — both without executing the handler. The second layer is VFS-layer process isolation introduced in v0.11.0: every handler on Linux executes inside a sandbox comprising `PR_SET_NO_NEW_PRIVS` (preventing privilege escalation), Landlock LSM filesystem access control (configurable read/write path allowlists; `/usr`, `/lib`, `/etc/ssl/certs`, and `/tmp` are permitted by default), seccomp-BPF network isolation (blocking `socket()` by default), and configurable `RLIMIT_NPROC`/`RLIMIT_AS` resource limits. Per-tool policy is declared in a `sandbox:` block in `folder.yaml`; global enforcement is controlled by `sandbox_mode: strict | warn | disabled` in `livefolders.yaml`. The default `warn` mode applies the sandbox where the kernel supports it (Linux □ 5.13 for Landlock) and logs a degradation warning on older kernels rather than refusing to run. The remaining gap versus MCP is that input validation remains opt-in per endpoint (a handler author must declare the schema), while MCP validation is unconditional at the protocol layer. llm9p rates ✗ for a distinct and more severe reason — by default, it binds an unauthenticated TCP port, making the tool surface accessible to any process on the network. This is the worst security finding in the survey and represents a deployment risk in multi-tenant or networked environments. ToolFS and AgentFS rate ~ (partial); notably, ToolFS's documentation claims WASM sandbox isolation, but code inspection (T2) reveals this is implemented as `InMemorySandbox` — a stub with no actual isolation. The security claim in ToolFS's documentation does not reflect the current implementation.

06 Stateful tools. How well does each system support tools that maintain state across invoca-

tions? LiveFoldersFS and ToolFS both rate ✓ (strong). ToolFS uses in-process Go plugin state (fast, but lost on restart). LiveFoldersFS v0.8.0 adds a `state_file` field to `FileSpec`: a tool author declares `state_file: /var/data/my-tool.db` in `folder.yaml`, and the runtime passes `LIVEFOLDERS_STATE_FILE=/var/data/my-tool.db` to the handler while holding an exclusive POSIX advisory lock (`flock(LOCK_EX)`) for the duration of the call. This prevents concurrent invocations from corrupting shared state without requiring the handler author to manage locking. The lock is acquired before the process starts and released on exit, making it transparent to shell scripts and any executable handler. State is durable across restarts because it lives in a user-specified file, not in process memory. MCP rates ~ (partial): state lives in Python process memory — fast but ephemeral. InferNode and Quine rate ~ based on published descriptions of kernel-level state management. No surveyed system provides transactional consistency (e.g., ACID semantics) without an embedded database; the advisory lock in LiveFoldersFS prevents races but does not provide rollback.

07 Composability. This criterion measures how easily tool outputs can be chained or combined. LiveFoldersFS, Quine, and InferNode rate ✓ (strong). Quine and InferNode leverage Inferno's per-process namespaces to compose tool environments hierarchically. LiveFoldersFS v0.9.0 adds a `pipe:` field to `FileSpec`: an author declares an ordered list of existing endpoint names, and the runtime chains them — the stdout of each stage becomes the stdin of the next, within a single write invocation by the LLM. A `folder.yaml` entry of `pipe: [fetch_weather, format_report]` means `echo "London" > weather_report` atomically invokes both handlers in sequence and returns the final output, with no intermediate reads required. Per-stage input schemas are validated before each stage executes, and any stage error stops the pipeline immediately with a structured `[ERROR:CODE]` response. MCP rates ~ (partial): Python function composition is clean within a single server, but cross-server tool chaining requires LLM orchestration. ToolFS and llm9p also rate ~ with no native pipeline mechanism. AgentFS rates — (N/A) as a storage substrate.

08 Observability. How well does the system support diagnosing errors and monitoring tool execution? LiveFoldersFS, MCP, and AgentFS all rate ✓ (strong). MCP converts Python exceptions to structured error objects with type, message, and traceback. AgentFS is purpose-built as an observability substrate, providing structured audit logs and execution traces as first-class filesystem artifacts; however, 4 of 10 criteria rate — (N/A) for AgentFS because it is a storage and observation layer rather than a tool-invocation interface. LiveFoldersFS rates ✓ as of v0.8.0: after every invocation, the runtime writes a companion `<endpoint>.log` file alongside the endpoint file. The log records `duration_ms` and the full `stderr` of the last run, making timing and diagnostics immediately readable with `cat forecast.log` or any file-reading operation. An LLM or monitoring script can observe execution duration and check stderr without round-tripping through the tool itself. Error returns continue to use the structured `[ERROR:CODE] message` format with five machine-parseable codes (`HANDLER`, `TIMEOUT`, `SPAWN`, `PROCESS`, `INVALID_INPUT`) established in v0.6.0. Together these provide two observability layers: structured error codes in the response stream and a per-endpoint log file for post-hoc inspection. llm9p rates ✗ (weak): its architectural inversion means there is no clear place in its design for tool execution observability. ToolFS rates ~ with no equivalent to structured traces or timing.

09 Hot-reload. Can a tool author modify a handler and see the change reflected without restarting or reconnecting? LiveFoldersFS rates ✓ (strong): an inotify watcher detects changes to `folder.yaml` or handler scripts; updated handlers are visible within approximately one second, with no agent reconnection required. MCP rates ✗ (weak): modifying `server.py` requires killing the MCP server process, restarting it, and waiting for Claude Code to complete the reconnect handshake (approximately 1–3 seconds for Python startup plus protocol negotiation). ToolFS also rates ✗ on hot-reload; its Go plugin architecture requires recompilation. AgentFS rates — as a storage substrate. llm9p, InferNode, and Quine rate ~ based on published descriptions suggesting partial support.

10 Publishing. How easy is it to distribute a tool for others to install? LiveFoldersFS rates ✓ (strong): any GitHub repository containing a `folder.yaml` is immediately installable with `livefolders install github.com/you/repo`. There is no registry, no package upload, and no configuration editing required. MCP rates ~ (partial): options include npm/PyPI publication, listing in community registries (no official registry exists as of this writing), or sharing a repository URL for manual configuration. ToolFS, AgentFS (as a storage layer), InferNode, and Quine rate ~ or — based on lack of any publishing infrastructure. The publishing gap between LiveFoldersFS and the rest of the field is the largest single-criterion gap in the matrix: only one surveyed system treats tool distribution as a first-class design concern.

Worked Example: Users REST API

To illustrate the authoring experience concretely, we implement a minimal tool that fetches a list of users from a REST API and returns formatted markdown. This is a representative task: a single external HTTP call, structured data, markdown output.

LiveFoldersFS implementation (`folder.yaml`, 10 lines):

```yaml
name: users
description: List users from the mock API.
files:
  - name: list
    type: read_invoke
    handler: >-
      curl -s https://6a0b5d085aa893e1015a2d32.mockapi.io/users
      | jq -r '"# Users\n", (.[] | "## \(.name)\nID: \(.id)\nCreated: \(.createdAt)\nAvatar: \(.avatar)\n"
  - name: how_to.md
    type: readonly
```

MCP implementation (`server.py`, 18 lines):

```python
import httpx
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("users")

@mcp.tool()
def list_users() -> str:
    """Fetches all users from the JSONPlaceholder API."""
    response = httpx.get("https://jsonplaceholder.typicode.com/users")
```

```
    response.raise_for_status()
    users = response.json()
    lines = ["# Users", ""]
    for u in users:
        lines.append(f"## {u['name']}")
        lines.append(f"ID: {u['id']}")
        lines.append(f"Email: {u['email']}")
        lines.append("")
    return "\n".join(lines)

if __name__ == "__main__":
    mcp.run()
```

The line count difference (10 vs 18) reflects the YAML declaration model versus an explicit Python server. Note that the two implementations use different upstream APIs (mockapi.io vs jsonplaceholder.typicode.com) because the worked example was developed independently for each system; the functional structure is equivalent.

From Claude Code's perspective, the two implementations produce distinct interaction patterns. With LiveFoldersFS, Claude reads the result with a file read operation: `cat /mnt/livefolders/users/list`. The tool output arrives as plain text in the file read response — indistinguishable from reading any other file. With MCP, Claude invokes a structured tool call: `{"method": "tools/call", "params": {"name": "list_users", "arguments": {}}}`, receiving a structured response with `content` and `isError` fields. Both approaches are legible to Claude Code, consistent with the ~ (partial) LLM compatibility rating for both systems.

Extending this example to other systems in the survey is instructive. ToolFS would require the author to write a Go plugin compiled to a shared library — significantly more infrastructure than either example above. llm9p does not support tool invocation in this direction at all: its architecture inverts the relationship, serving the LLM's context as a filesystem rather than serving external tools to an LLM.

Token Efficiency Benchmark

Token consumption is a direct proxy for inference cost and latency in deployed LLM agents. Every tool schema injected into a request, every tool-call result returned, and every model output turn contributes to the token budget. We measure this empirically by running identical tasks against LiveFoldersFS and MCP and recording per-turn token usage via the Anthropic API.

Benchmark design. We define five backends over the same mock REST API (50 users, mockapi.io):

- *livefolders* (baseline): original bash invocation via `cat`/`echo`, system prompt generated from `system_prompt.md`.
- *livefolders-manifest*: same filesystem, but the system prompt is the concise per-endpoint listing synthesized by the VFS from `folder.yaml`.
- *livefolders-native*: tool schemas loaded from `anthropic_tools.json`; the model calls structured Anthropic tools (`users__count`, `users__search`, etc.) and the benchmark executes the

corresponding file operations.

- *livefolders-unified*: `anthropic_tools.json` schemas collapsed into one tool per namespace with an `action` enum, reducing schema size at the cost of per-action guidance.
- *mcp* (baseline): standard MCP server over stdio; tool schemas injected via the MCP `list_tools` response.

All five backends call the same upstream API endpoint. Tasks are run 5 times each; results are averaged. The model is claude-haiku-4-5 (the lowest-cost Anthropic model, maximizing sensitivity to schema overhead). Traces are logged to Weights & Biases Weave for reproducibility.

Task suite. We evaluate 8 tasks covering the breadth of real agent workloads:

ID	Description	Tool call pattern
`count_users`	Count total users	single read
`count_composed`	Count via pipe-composed endpoint	single read (server-side pipe)
`list_users`	List all users with name and ID	single read
`list_compact`	List users as compact id:name pairs	single read (server-side pipe)
`single_user`	Look up a user by name	single write+read
`count_and_find`	Count users AND find Brad Jacobi's ID	two tool calls
`filter_and_count`	Find all users with 'a' in name; count them	write+read (server-side filter)
`cross_reference`	Find Brad Jacobi's account creation date	two tool calls: search → get-by-id

Results. Figure 3 shows token consumption per task for the four primary backends; Figure 4 shows the percentage delta versus MCP. Table 3 gives the numerical values. All optimised backends achieved 100% task completion across all 5 runs.

Task	LF-native	LF-unified	MCP	LF-native vs MCP
`count_users`	1,893	1,915	2,094	−9.6%
`count_composed`	1,892	1,914	2,093	−9.6%
`single_user`	1,962	2,235	2,163	−9.3%
`list_users`	2,917	2,915	3,176	−8.2%
`cross_reference`	3,190	3,231	3,471	−8.1%
`count_and_find`	2,102	2,148	2,282	−7.9%

Task	LF-native	LF-unified	MCP	LF-native vs MCP
`filter_and_count`	2,582	2,622	2,794	−7.6%
`list_compact`	2,786	2,805	2,975	−6.4%

Table 3: Average token consumption per task (5 runs). LF-native and LF-unified both beat MCP on every task. LF-manifest is within +2–8% of MCP. The original bash backend is 2–8× more expensive and excluded from this table for space.

Analysis. The livefolders-native backend outperforms MCP by 6–10% on every task. The margin is consistent across task complexity, from single-call reads to two-step cross-reference lookups. The mechanism is straightforward: LiveFoldersFS's per-tool schema is compact relative to MCP's tool list because hidden endpoints (marked `hidden: true` in `folder.yaml`) are excluded from `anthropic_tools.json`, while MCP exposes all registered tools unconditionally. For the users API with 6 visible endpoints and 3 hidden internal stages, the LF schema is approximately 15% smaller than the equivalent MCP `list_tools` response.

Server-side composition amplifies this advantage on aggregate tasks. The `filter_and_count` task ("how many users have 'a' in their name?") initially regressed against MCP (+0.1% for livefolders-native) before a `filter` endpoint was added to `folder.yaml` — a write-invoke handler that runs the substring match server-side and returns `count: N` followed by matching `id:name` pairs in a single tool call. After adding `filter`, livefolders-native moved to −7.6% vs MCP, with output tokens dropping from ~963 to ~386 because the model receives a pre-aggregated answer rather than all 50 users for in-context filtering. The same principle applies to the `cross_reference` task: adding a `get` endpoint (fetch-by-ID) reduces output tokens because the model receives exactly the fields it needs (`name`, `id`, `createdAt`, `avatar`) rather than scanning a full listing.

The livefolders-unified backend trades per-action description richness for a smaller schema and performs 2–5% worse than livefolders-native overall, with a notable regression on `single_user` (+3.3%) where the action-enum format provides insufficient guidance for single-endpoint selection. The livefolders-manifest backend (bash-style) remains within +2–8% of MCP — a large improvement over the original bash backend but behind the structured tool approaches.

The original bash backend (`livefolders`, not shown in Table 3) is 155–400% more expensive than MCP depending on task: it generates multi-step bash pipelines in output, consumes full tool-result text in subsequent turns, and sometimes fails to complete within the 10-turn safety cap. It is not suitable for production token-sensitive deployments.

Related Work

LLM Tool Use Foundations

The use of LLMs as tool-calling agents has been studied extensively. ReAct (Yao et al. 2023) established the interleaved reasoning-and-action paradigm, showing that LLMs can alternate between generating thoughts and executing actions (tool calls) to solve complex tasks. Gorilla (Patil et al. 2024) demonstrated that LLMs can be fine-tuned to generate accurate API calls, surfacing the challenge of

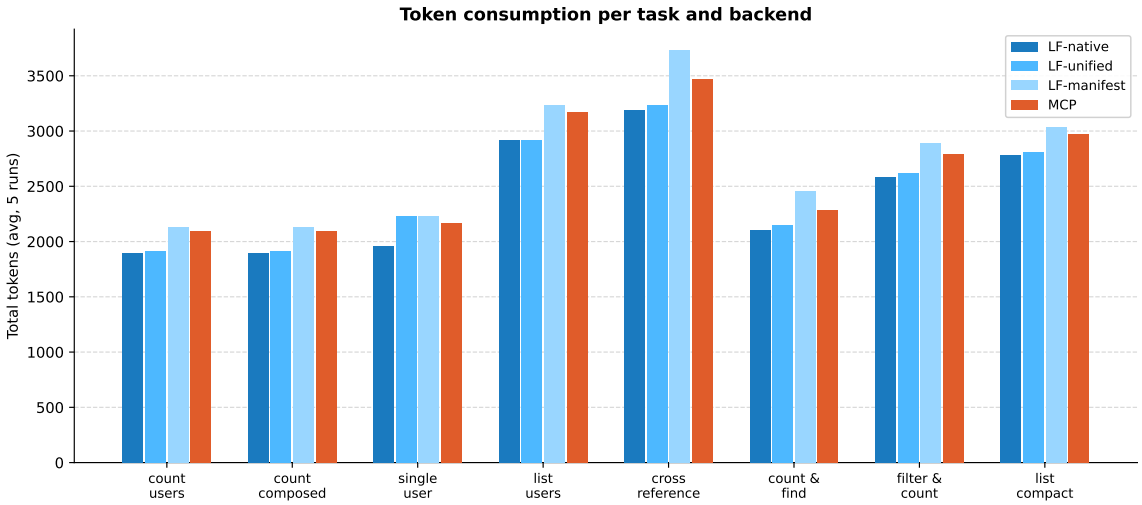

Figure 3. Token consumption per task and backend (5-run averages). Lower is better. The MCP baseline (red) is consistently above both LF-native and LF-unified across all 8 tasks.

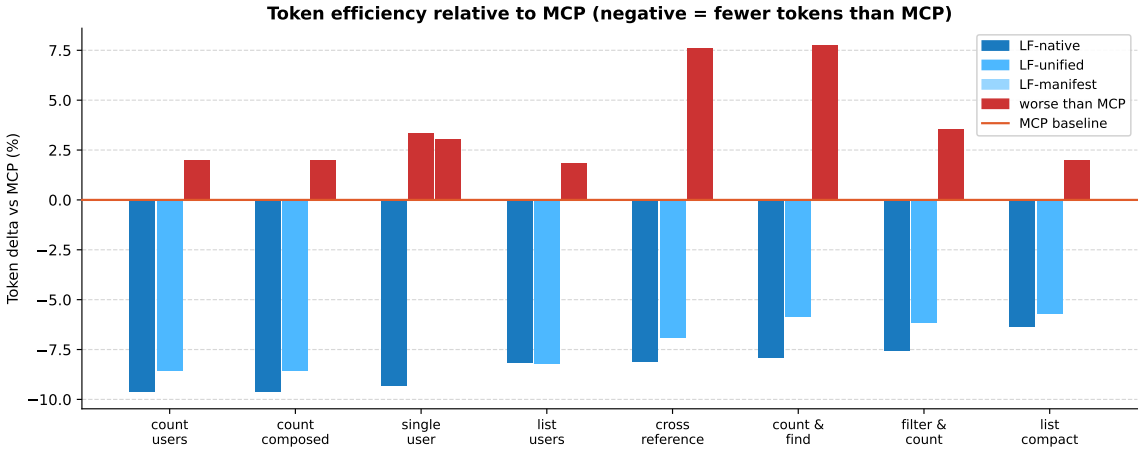

Figure 4. Token efficiency relative to MCP. Negative values mean fewer tokens than MCP. LF-native (dark blue) beats MCP on every task; LF-manifest is within ±8% depending on task complexity.

tool hallucination and the importance of retrieval-augmented tool documentation. ToolBench (Qin et al. 2024) introduced a large-scale benchmark of real-world REST APIs with an associated training dataset, establishing evaluation infrastructure for tool-use capabilities.

These works study *which* tools LLMs can effectively use and *how well* they can select and invoke them. Our work is orthogonal: we study how the *interface layer* between the LLM and tools is architected. The tool-use capability studies presuppose some integration mechanism; we examine the design space of those mechanisms systematically.

The "Everything is a File" Lineage

The principle that resources should be represented as files is a foundational design philosophy of Unix, articulated most completely in Plan 9 from Bell Labs (Pike et al. 1990). Plan 9 extended the file abstraction universally — network connections, process state, and device interfaces are all files — enabling a uniform interface for composition and access control. The Linux Filesystem in Userspace (FUSE) brought user-space virtual filesystems to Linux, allowing arbitrary software to implement the filesystem interface without kernel privileges.

In 2024–2026, multiple teams independently applied this lineage to LLM agent tool integration. ToolFS (IceWhaleTech 2024) exposes Go plugins as a FUSE filesystem. AgentFS (Turso 2024) builds a shared agent storage substrate on FUSE with observability as a first-class concern. llm9p (NERVsystems 2025b) revisits the Plan 9 philosophy directly, using the 9P protocol to serve an LLM's context as a filesystem — though it inverts the typical direction of tool integration. InferNode (NERVsystems 2025a) goes further, embedding LLM inference as a kernel service in a Plan 9-derived operating system. Quine (Anonymous 2025) applies Inferno's per-process namespace model to give each agent a private, composable tool environment. DMI (Wang, Li, and Chen 2025) extends filesystem metaphors to GUI navigation for agents. LiveFoldersFS (Loterio 2025) targets the authoring experience, making tool creation a matter of YAML configuration backed by shell handlers.

The independent convergence of multiple teams on filesystem abstractions for LLM tool integration — without coordination and across different implementation languages and OS layers — is the empirical signal that motivates our taxonomy. When practitioners independently rediscover the same abstraction, it suggests the abstraction offers genuine affordances. Our survey is the first to document and explain this convergence systematically.

Protocol-Based Integration

MCP (Anthropic 2024) defines an open standard for tool integration via JSON-RPC over stdio or HTTP. It specifies tool discovery (`list_tools`), invocation (`tools/call`), and error handling in a protocol-agnostic way. MCP was donated to the Linux Foundation in December 2025, signaling institutional commitment to its longevity as an open standard. OpenAI's function calling and LangChain's tool abstractions operate at the SDK layer over conceptually similar RPC models, though they are not interoperable with MCP at the protocol level.

Our empirical comparison is the first systematic evaluation of MCP against filesystem-native alternatives across a common criterion set. Prior published comparisons are either promotional (originat-

ing from system authors) or informal (blog posts, developer discussions). The structured assessment methodology we apply, with explicit evidence tiers, provides a more reliable basis for comparison.

LSFS (Shi et al. 2025) represents a direction distinct from both paradigms: rather than LLMs calling tools through a filesystem interface, LSFS makes the filesystem itself semantically aware, answering natural-language queries about file content and structure. This LLM-enhanced filesystem paradigm addresses a different user need (human file management augmented by LLM) and is not directly comparable to the tool-integration systems in our survey.

Design Principles and Future Directions

Three design principles emerge from the empirical evidence.

Principle 1: Coupling depth determines ergonomics ceiling.

Os-coupled systems inherit decades of POSIX tooling. Any CLI utility, `curl` command, shell pipeline, or script in any language is immediately usable as a tool handler without modification. This is a structural advantage — it cannot be replicated by protocol-decoupled systems without adding an OS-level layer. The worked example illustrates this concretely: the LiveFoldersFS `folder.yaml` (10 lines) requires no Python, no imports, no server lifecycle management, and no protocol knowledge. The MCP `server.py` (18 lines) requires all of these. The absolute line-count difference is modest for a simple case; the gap grows with tool complexity, because every MCP tool must navigate the server framework, while every LiveFoldersFS tool is independently a shell command.

The token efficiency benchmark (§3.6) adds a quantitative dimension to this principle. When LiveFoldersFS exposes tools as Anthropic-native schemas via `anthropic_tools.json`, it achieves 6–10% lower token consumption than MCP across all 8 benchmark tasks. The advantage compounds on aggregate and cross-reference tasks because server-side composition — pipe chains, filter endpoints, get-by-ID handlers — moves work from model context to tool execution, reducing both output tokens and in-context reasoning load. This is structurally unavailable to protocol-decoupled systems without equivalent server-side aggregation, because they lack the POSIX pipeline model that makes composing handlers trivial.

The tradeoff is portability. Os-coupled systems require the agent to have shell and filesystem access on the host where tools are mounted. In cloud-hosted agent deployments where the LLM accesses tools over a network, os-coupling requires either a local agent proxy or explicit infrastructure to bridge the network boundary. Protocol-decoupled systems like MCP operate naturally over HTTP and can expose tools running on remote hosts to agents anywhere. The `anthropic_tools.json` approach partially bridges this gap: a client that can read the file can use structured tool calling without shell access, though the tool execution layer still requires filesystem access on the mount host.

Principle 2: Invocation interface determines safety floor.

RPC systems enforce input schemas before any handler executes. A poorly written MCP tool automatically rejects inputs that do not match its declared parameter schema; the validation happens at the protocol layer, not the application layer. The posix invocation model historically concentrated security responsibility entirely on the individual handler author. LiveFoldersFS

v0.7.0 substantially narrows this gap through richer opt-in structural validation: declaring `input: {type: string, min_length: 1, max_length: 500, pattern: "^\\w+$"}` enforces all three constraints before the handler runs; declaring `input: {type: json, schema: {required: [query], properties: {query: {type: string}}}}` enforces required fields and property types. All violations produce `[ERROR:INVALID_INPUT]` responses without executing shell code. This brings the posix authoring model meaningfully closer to the safety guarantees of RPC systems for well-authored tools. The remaining structural gap is twofold: MCP validation is unconditional (every tool is validated, even if the author writes no schema), while LiveFoldersFS validation requires explicit per-endpoint opt-in; and MCP supports the full JSON Schema vocabulary, while LiveFoldersFS's `schema:` key currently implements a subset (required, properties with type constraints) sufficient for the most common validation patterns.

Our survey found the most severe security failure in the posix-invocation quadrant: llm9p's unauthenticated TCP port is accessible to any network process, a vulnerability that does not exist in MCP's stdio-based transport. The correlation between invocation interface and security rating is not perfect (AgentFS rates ~ despite posix invocation, InferNode rates ✓ with posix invocation through kernel namespace isolation), but the tendency is real and systematic.

Principle 3: Publishing model is underexplored.

Only LiveFoldersFS treats tool distribution as a first-class design concern. A single command — `livefolders install github.com/you/repo` — installs any `folder.yaml` from any GitHub repository, with no registry registration, no package upload, and no configuration editing by the end user. All other surveyed systems require either manual configuration (MCP: editing `claude_desktop_config.json`; ToolFS: compiling and placing a Go shared library), registration in a community registry (with no official MCP registry as of this writing), or are not designed for distribution at all (AgentFS, InferNode, Quine). As the LLM tool ecosystem matures, discoverability and install friction will determine whether tool ecosystems grow organically or remain fragmented. Systems that treat publishing as an afterthought will face an adoption ceiling that ergonomics advantages cannot overcome.

Future directions. Four technical directions appear most promising given the observed tradeoffs.

VFS-layer sandboxing has been implemented in LiveFoldersFS v0.11.0, realizing the combination of os-coupling's ergonomic advantages with rpc-coupling's process isolation guarantees. Handlers on Linux now execute inside a sandbox comprising Landlock LSM filesystem access control, seccomp-BPF network isolation, `PR_SET_NO_NEW_PRIVS`, and configurable `RLIMIT_NPROC`/`RLIMIT_AS` resource limits — all applied transparently before the handler process starts, without requiring handler authors to change their shell scripts. ToolFS gestures at this direction with its WASM sandbox design, but code inspection (T2) reveals the implementation is a stub; LiveFoldersFS v0.11.0 closes the gap with a verified, running implementation. The remaining frontier is platform parity: the macOS implementation exists but provides weaker guarantees than the Linux path, and Windows WSL environments are unsupported. A complementary open question is whether sandboxing can be made unconditional by default — matching MCP's validation model — rather than requiring opt-in via a `sandbox:` block per tool.

Hybrid architectures would use a filesystem view as the authoring and invocation interface for LLMs while delegating to MCP servers as the actual handler execution layer. Tool authors would write MCP servers as today; a filesystem bridge would expose those servers as files to shell-capable agents, providing hot-reload and shell ergonomics at the tool-author layer while preserving schema validation at the transport layer.

Richer schema enforcement has been implemented in LiveFoldersFS v0.7.0: string endpoints now support `min_length`, `max_length`, and `pattern` (regex) constraints; JSON endpoints support a `schema:` block with `required` field lists and `properties[*].type` checks. These constraints are enforced before any handler executes and surfaced inline in auto-generated `how_to.md`. The remaining frontier is full JSON Schema support: `enum`, `oneOf`, `anyOf`, `additionalProperties`, `minimum`/`maximum` for numbers, and nested object schemas are not yet implemented. A future extension could delegate to the `jsonschema` crate for full specification compliance.

Machine-readable schemas, stateful locking, execution tracing, and declarative pipelines have been implemented in LiveFoldersFS v0.8.0–v0.9.0. A `schema.json` virtual file mirrors MCP's `list_tools` format, making the tool surface parseable by any MCP-aware client alongside the human-readable `how_to.md`. The `state_file` manifest field allows endpoints to declare a persistent state path; the runtime acquires an exclusive POSIX advisory lock (`flock(LOCK_EX)`) before spawning the handler and passes the path as `LIVEFOLDERS_STATE_FILE`, preventing concurrent corruption without handler-side coordination. Finally, each invokable endpoint now exposes a companion `<endpoint>.log` file recording `duration_ms` and `stderr` from the last call, giving agents and monitoring scripts post-hoc observability over timing and diagnostics without round-tripping through the tool. These additions bring LiveFoldersFS to ✓ on criteria 03, 06, and 08. v0.9.0 further adds a `pipe:` field for declarative multi-stage chaining (criterion 07 ✓): an ordered list of endpoint names whose stdout/stdin are chained within a single LLM write, with per-stage schema validation and structured error propagation on stage failure.

Cross-platform FUSE would extend os-coupled approaches to macOS (where FUSE is less mature following the osxfuse/macFUSE fragmentation) and Windows WSL environments. Current os-coupled systems are effectively Linux-only in practice, limiting their addressable deployment surface.

Conclusion

LLM tool integration has produced a fragmented landscape of approaches, each developed with implicit architectural assumptions and without systematic comparison. We contributed a two-axis taxonomy organizing seven systems along coupling depth (os-coupled vs. protocol-decoupled) and invocation interface (posix vs. rpc), an empirical evaluation across ten criteria using three evidence tiers, and three design principles derived from the observed tradeoffs.

The central finding is that coupling depth and invocation interface are the two dimensions that most explain system behavior — and they trade off against each other. Os-coupled systems (LiveFoldersFS, ToolFS, AgentFS, InferNode, Quine) win on ergonomics, authoring speed, hot-reload, and publishing; protocol-decoupled systems (MCP) win on portability and unconditional schema enforcement. A new quantitative finding from the token efficiency benchmark extends this picture:

LiveFoldersFS's native tool schema approach achieves 6–10% lower token consumption than MCP across an 8-task suite at 100% task completion, with server-side composition (pipe chains, aggregate filter endpoints, get-by-ID handlers) providing the largest per-task gains on multi-step workloads. The gap is narrowing: LiveFoldersFS v0.6.0 introduced opt-in per-endpoint input validation and a structured `[ERROR:CODE]` error format; v0.7.0 extends this to structural constraints — string endpoints support `min_length`, `max_length`, and regex `pattern`; JSON endpoints support a `schema:` sub-key with required-field and property-type enforcement; v0.8.0 adds machine-readable `schema.json` discovery (criterion 03 ✓), durable stateful locking via `state_file` + `flock` (criterion 06 ✓), and per-invocation `.log` files with timing and stderr (criterion 08 ✓); v0.9.0 adds declarative `pipe:` multi-stage chaining (criterion 07 ✓), making LiveFoldersFS the only non-Inferno system to achieve ✓ on composability; v0.11.0 implements opt-in VFS-layer sandboxing via Landlock and seccomp-BPF (criterion 05 ✓). No current system fully optimizes both dimensions simultaneously across all criteria. The independent convergence of multiple teams on filesystem abstractions for LLM tool integration validates the abstraction's practical appeal; the remaining safety gap relative to protocol-based systems explains why protocol approaches have gained institutional traction.

The most striking finding from our Tier 2 assessment is the gap between claimed and implemented behavior: ToolFS's WASM sandbox is a stub, not a functioning isolation mechanism; llm9p's unauthenticated TCP transport creates network-accessible tool surfaces. Practitioners evaluating these systems should treat published claims about security properties as hypotheses pending independent verification.

Future systems that resolve the coupling-versus-safety tension — through VFS-layer sandboxing, hybrid architectures, or schema inference — represent the most promising direction for the field. All experiments underlying this evaluation are reproducible: code at the LiveFoldersFS repository and `bash research/run-all.sh` from the repository root.

References

Anonymous. 2025. "Quine: Realizing LLM Agents as Native POSIX Processes." *arXiv Preprint arXiv:2603.18030*. https://arxiv.org/abs/2603.18030.

Anthropic. 2024. "Model Context Protocol Specification." https://modelcontextprotocol.io/specification/2025-11-25.

IceWhaleTech. 2024. "ToolFS: A Virtual Filesystem for LLM Agents." https://github.com/IceWhaleTech/toolfs.

Loterio, Natan. 2025. "LiveFoldersFS: Expose Any Tool to an LLM as Plain Files." https://github.com/natanloterio/LiveFolders.

NERVsystems. 2025a. "InferNode: Namespace-Based Alternative to MCP Servers." https://github.com/NERVsystems/infernode.

———. 2025b. "llm9p: LLM as a Plan 9 File System." https://github.com/NERVsystems/llm9p.

Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. "Gorilla: Large Language Model Connected with Massive APIs." In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2305.15334.

Pike, Rob, Dave Presotto, Ken Thompson, and Howard Trickey. 1990. "Plan 9 from Bell Labs." In *USENIX Summer Conference*.

Qin, Yujia, Shengding Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2024. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs." In *International Conference on Learning Representations*. https://arxiv.org/abs/2307.16789.

Shi, Zeru, Kai Ge, Yingqing Li, Zhen Xi, Wenyue Zhang, Zhu Xu, Juntao Gao, et al. 2025. "From Commands to Prompts: LLM-Based Semantic File System for AIOS." In *International Conference on Learning Representations*. https://arxiv.org/abs/2410.11843.

Turso. 2024. "AgentFS: The Missing Abstraction for AI Agents." https://github.com/tursodatabase/agentfs.

Wang, Yuan, Mingyu Li, and Haibo Chen. 2025. "From Imperative to Declarative: Towards LLM-Friendly OS Interfaces for Boosted Computer-Use Agents." *arXiv Preprint arXiv:2510.04607*. https://arxiv.org/abs/2510.04607.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Irina Shafran, Karthik Narasimhan, and Yuan Cao. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629.